Pablo Caricio

ECO VISION AI - GUIA TÉCNICO E DIDÁTICO

Código, conceitos, decisões, experimentos e interpretação dos resultados

Projeto: EcoVision AI

Disciplina: Deep Learning and Vision - Computer Vision

Versão final - guia técnico consolidado após validação local e execução integral dos quatro notebooks no Google Colab com GPU Tesla T4.

2026

Como usar este documento

Este guia foi construído para estudo, não apenas para entrega. A proposta é permitir que o projeto seja revisado do início ao fim, conectando cada módulo de código aos conceitos de deep learning e visão computacional que ele implementa. Recomenda-se estudar em ordem: fundamentos comuns, ViT, CLIP, CNN, raio-X/cGAN e, por fim, o glossário e as perguntas de revisão.

Status final de reprodutibilidade: os principais experimentos foram executados localmente, os checkpoints foram revalidados após a reorganização do projeto e os quatro notebooks foram executados integralmente no Google Colab com GPU Tesla T4. O A1 reproduziu o ViT scratch/pré-treinado e attention maps; o A2 reconstruiu o corpus ADS-16 e executou CLIP em CUDA; o A3 reconstruiu o dataset deduplicado e reproduziu 98,49% de accuracy; e o A4 reproduziu o baseline de raio-X, as três cGANs, a geração sintética e a queda do COVID recall de 55,56% para 38,89%. Assim, o guia pode ser estudado junto com os notebooks e os artefatos de resultados sem necessidade de retreinar os modelos já preservados nos checkpoints.

1. Arquitetura do projeto

```
EcoVision-AI-organized/
├── src/
│   ├── common/
│   ├── vit/
│   ├── clip/
│   ├── cnn/
│   └── xray/
├── notebooks/
├── data/
├── models/
├── results/
└── reports/
```

A reorganização separa cada atividade por domínio. O princípio é manter dados, modelos, treinamento, avaliação e geração próximos do problema correspondente. Isso reduz ambiguidade de nomes genéricos e facilita estudar o fluxo de cada experimento.

Pasta	Responsabilidade
src/vit	Attention, Transformer Encoder, ViT scratch, ViT pré-treinado e attention maps.
src/clip	Download/preparação do ADS-16, embeddings, ranking, threshold e busca semântica.
src/cnn	Limpeza/deduplicação, pipeline de dados, EfficientNet-B0, treino e avaliação.
src/xray	Preparação médica, ResNet-18, cGANs, geração sintética e comparações.
models	Checkpoints .pth/.pt; não versionados no Git.
results	JSONs, curvas, matrizes de confusão, previews e evidências.
notebooks	Entregáveis A1-A4.1 reproduzíveis.

2. Fundamentos comuns do código PyTorch

2.1 Tensor, batch e shape

PyTorch representa imagens e parâmetros como tensores. Uma batch RGB típica tem shape [B, 3, H, W]. B é a quantidade de exemplos processados de uma vez; 3 são os canais; H e W são altura e largura. No projeto, classificadores trabalham principalmente com 224x224; a GAN trabalha em grayscale 64x64.

```
images.shape  # [batch, channels, height, width]
labels.shape  # [batch]
```

2.2 Forward, logits, loss, backward e optimizer

O ciclo básico de treinamento aparece em ViT, CNN e ResNet. O forward transforma entradas em logits. Logits são scores não normalizados. A CrossEntropyLoss compara logits com a classe real. loss.backward() usa backpropagation para calcular gradientes. optimizer.step() altera os parâmetros. optimizer.zero_grad() limpa gradientes acumulados antes do próximo batch.

```
optimizer.zero_grad()
logits = model(images)
loss = criterion(logits, labels)
loss.backward()
optimizer.step()
```

Durante avaliação, model.eval() desativa comportamentos de treino, como Dropout, e torch.inference_mode() evita construção do grafo de gradientes, reduzindo memória e custo.

2.3 AdamW e weight decay

AdamW combina adaptação individual de learning rates com weight decay desacoplado. O weight decay funciona como regularização, penalizando parâmetros muito grandes. No ViT scratch foi usado weight decay 0,05; nos modelos pré-treinados, valores menores preservam melhor o conhecimento inicial.

2.4 Early stopping e checkpoints

Early stopping encerra o treinamento quando a validation loss deixa de melhorar por um número definido de épocas. O checkpoint salva state_dict do modelo, época e métricas. O objetivo é avaliar o melhor estado observado, e não simplesmente o último epoch.

```
if val_loss < best_val_loss:
    torch.save({"model_state_dict": model.state_dict(), ...}, checkpoint_path)
else:
    epochs_without_improvement += 1
```

2.5 Seed e reprodutibilidade

A seed 42 é usada em diferentes etapas para reduzir variação aleatória: splits, amostragem, inicialização e geração. Uma seed melhora reprodutibilidade, mas não garante bitwise determinism entre GPUs, bibliotecas e versões diferentes.

3. A1 - Vision Transformers: código e conceitos

3.1 src/vit/models/attention.py

Este módulo implementa ScaledDotProductAttention e MultiHeadAttention. É o ponto mais fundamental da atividade porque não depende de uma implementação pronta de nn.MultiheadAttention.

Elemento	Responsabilidade
ScaledDotProductAttention.forward	Recebe Q, K e V; calcula QK^T/sqrt(d_k), softmax e combinação com V; retorna saída e matriz de atenção.
MultiHeadAttention.__init__	Cria projeções lineares independentes para Q, K e V e a projeção de saída.
MultiHeadAttention.forward	Projeta tokens, separa heads, executa attention em paralelo, concatena e projeta de volta para embed_dim.
testar_*	Executam smoke tests de shapes, ausência de NaN e consistência estrutural.

```
scores = (Q @ K.transpose(-2, -1)) / sqrt(d_k)
weights = softmax(scores, dim=-1)
output = weights @ V
```

Conceito-chave: uma head aprende um padrão de relação; múltiplas heads permitem que a mesma sequência seja projetada em subespaços diferentes. No EcoVision, embed_dim=192, heads=3 e d_head=64.

3.2 src/vit/models/transformer.py

FeedForward é um MLP aplicado token a token. TransformerEncoderBlock combina LayerNorm, Multi-Head Attention, residual connection, novo LayerNorm e FeedForward. A implementação é pre-norm: normaliza antes da subcamada.

```
x = x + attention(LayerNorm(x))
x = x + feed_forward(LayerNorm(x))
```

Residual connections ajudam o fluxo de gradientes e preservam a informação original. LayerNorm estabiliza a distribuição das representações dentro do token. GELU fornece não linearidade no FFN.

3.3 src/vit/models/vit.py

PatchEmbedding transforma a imagem em sequência de patches por convolução com kernel=stride=patch_size. Para 224x224 e patch 16, obtemos 14x14=196 patches. ViTEmbeddings adiciona um CLS token aprendível e positional embedding, resultando em 197 tokens.

```
224 / 16 = 14
14 x 14 = 196 patches
196 + 1 CLS = 197 tokens
shape = [B, 197, 192]
```

O positional embedding é necessário porque a operação de self-attention não tem noção intrínseca de ordem/posição. Ele injeta informação espacial para diferenciar, por exemplo, um patch do canto superior de um patch do centro.

3.4 src/vit/models/vit_model.py

EcoVisionViT compõe embeddings, quatro TransformerEncoderBlocks, LayerNorm final e classification head. O vetor do CLS token final é usado como representação global da imagem e projetado para nove logits.

Hiperparâmetro	Valor
image_size	224
patch_size	16
embed_dim	192
num_heads	3
num_layers	4
mlp_dim	768
dropout	0,1
num_classes	9

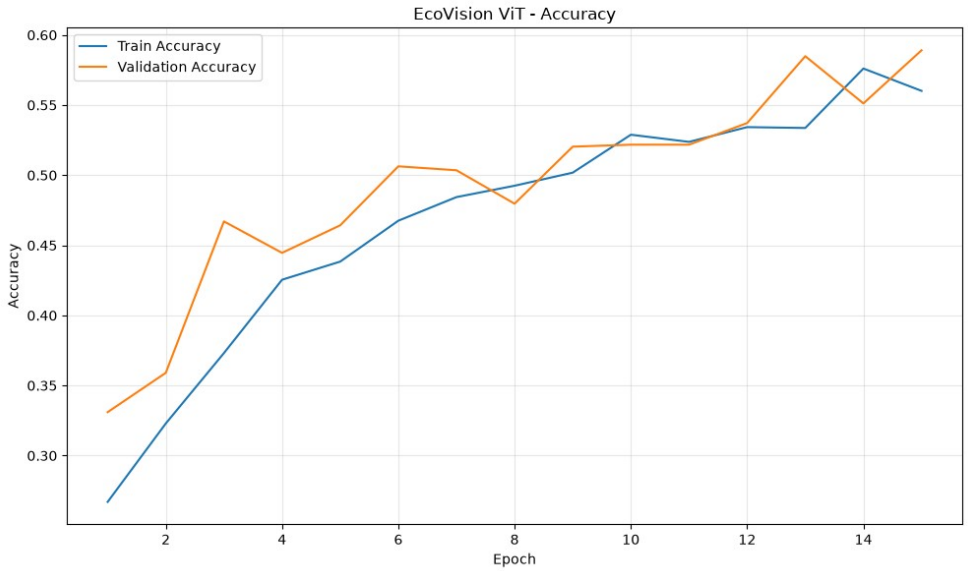
3.5 src/vit/data_pipeline.py e download_dataset.py

download_dataset.py prepara o RealWaste. data_pipeline.py localiza o dataset, cria índices estratificados e constrói datasets/loaders. Separar transform de treino e avaliação é fundamental: augmentation aleatória entra apenas no treino; validação e teste precisam ser estáveis.

3.6 src/vit/train_scratch.py

Função	O que faz
treinar_epoca	Executa forward, CrossEntropyLoss, backward e AdamW em todos os batches de treino.
avaliar	Executa inferência em validação e calcula loss/accuracy sem atualizar pesos.
salvar_checkpoint	Persiste melhor estado quando validation loss melhora.
salvar_historico	Grava métricas por época em JSON.
gerar_graficos	Produz curvas de loss e accuracy.
main	Orquestra dataset, modelo, treino, early stopping e salvamento.

Resultado: melhor checkpoint na época 13; teste 55.26% accuracy. O resultado é importante para estudo porque comprova que uma arquitetura correta pode ainda apresentar desempenho limitado quando treinada do zero com dados relativamente escassos.



Exemplo visual - curvas de accuracy do ViT scratch.

3.7 src/vit/attention_maps.py

O módulo carrega o checkpoint, encontra um exemplo corretamente classificado, extrai a atenção do CLS token para os 196 patches, reorganiza os valores em grade 14x14, interpola para o tamanho da imagem e salva heatmaps/overlays. No exemplo de Plastic analisado, o head 0 distribui atenção por várias regiões internas do objeto; o head 1 enfatiza contornos e a região inferior, com alguma resposta ao fundo; e o head 2 concentra pesos em regiões internas de maior contraste. A principal lição de estudo é que heads diferentes podem aprender relações espaciais diferentes. Ainda assim, attention weight não equivale automaticamente a causalidade ou explicabilidade completa da decisão.

3.8 src/vit/models/pretrained_vit.py e train_pretrained.py

EcoVisionPretrainedViT encapsula vit_tiny_patch16_224 da timm, troca o head e oferece congelar_backbone/descongelar_modelo. train_pretrained.py executa duas fases: head-only por 2 epochs (LR 1e-3) e fine-tuning por até 8 epochs (LR 1e-5).

O teste alcançou 86.54% accuracy, ganho de 31.28 pp sobre o scratch. Conceitualmente, feature extraction preserva a representação prévia; fine-tuning permite adaptar as features com passos menores.

3.9 BERT versus ViT: o que cada pré-treinamento otimiza

BERT e ViT reutilizam a ideia de Transformer Encoder, mas o objetivo de pré-treinamento é diferente. No BERT original, o Masked Language Modeling (MLM) tenta maximizar a probabilidade dos tokens ocultos dados os tokens de contexto em ambas as direções; o artigo original também inclui Next Sentence Prediction (NSP). No ViT original, os patches funcionam como tokens visuais e o modelo é pré-treinado de forma supervisionada para minimizar a loss de classificação em grandes datasets rotulados. Para estudo: BERT aprende estrutura contextual da linguagem prevendo conteúdo ausente; ViT aprende representações visuais discriminativas a partir do erro de classificação e depois é adaptado ao domínio alvo.

4. A2 - CLIP: código e conceitos

4.1 src/clip/prepare_corpus.py

O módulo localiza usuários/imagens, calcula SHA-256, remove duplicatas exatas, seleciona cinco imagens por usuário e salva um manifest CSV. O manifest registra caminho, usuário, sentimento e hash, permitindo rastreabilidade do corpus de 600 imagens.

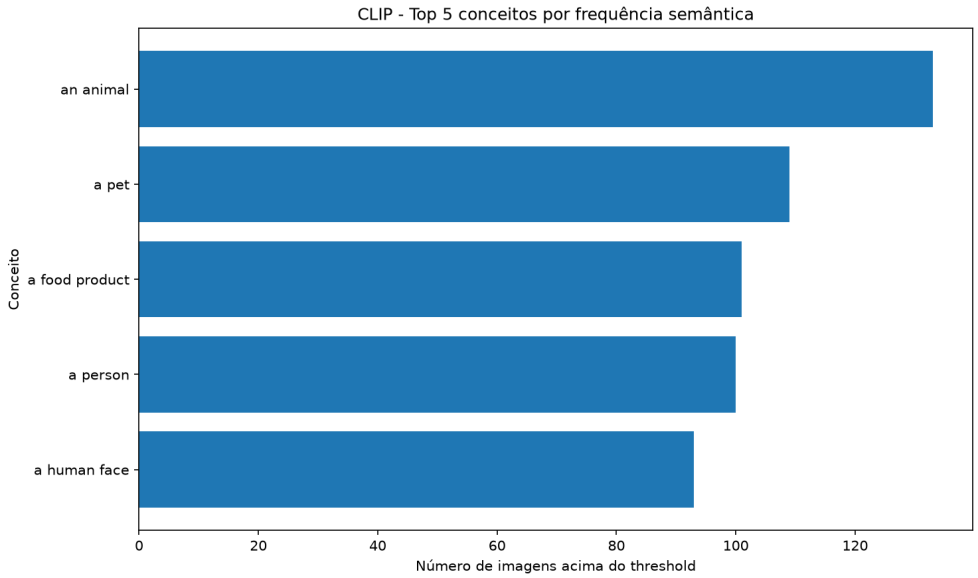
Função	Responsabilidade
calcular_sha256	Cria hash do conteúdo do arquivo para detectar igualdade exata.
localizar_usuarios	Encontra pastas de usuários no dataset.
remover_duplicatas	Mantém um representante por hash.
selecionar_imagens	Seleciona corpus representativo com seed fixa.
copiar_corpus	Materializa o subconjunto em data/ads16_clip.
salvar_manifest	Persiste metadados e caminhos.

4.2 src/clip/semantic_ranking.py

É o pipeline principal de embeddings. carregar_clip instancia CLIPModel/CLIPProcessor; gerar_embeddings_texto codifica 24 conceitos; gerar_embeddings_imagens percorre o corpus em batches de 16; normalizar_embeddings aplica L2; calcular_similaridades executa produto matricial; gerar_ranking conta scores acima do threshold e calcula médias/máximos.

```
image_embeddings = image_embeddings / ||image_embeddings||
text_embeddings  = text_embeddings / ||text_embeddings||
similarities = image_embeddings @ text_embeddings.T
```

O threshold adotado é 0,22. O Top-5 foi: an animal, a pet, a food product, a person, a human face. O score não é probabilidade; é proximidade no espaço multimodal.



Exemplo visual - ranking dos cinco conceitos mais frequentes.

4.3 src/clip/threshold_analysis.py

Esse módulo compara cortes 0,22, 0,23, 0,24 e 0,25. O número de pares cai de 1.254 para 158. A lição é que um threshold deve ser calibrado pela distribuição do corpus; pequenas mudanças no corte alteram muito a cobertura.

4.4 src/clip/semantic_search.py

A busca gera embedding de uma consulta, compara com embeddings visuais já salvos, ordena scores e retorna Top-5. O módulo também cria slug de nome, visualização e JSON/CSV. Reutilizar embeddings de imagens evita recalcular a parte mais cara em cada consulta.

Consulta	Leitura do resultado
a person	Parcialmente boa: recupera pessoas e representações humanas, mas inclui ruído contextual.
an animal	Muito boa: os resultados recuperados são fortemente coerentes com o conceito de animal.
a food product	Boa com ruído: os primeiros resultados são adequados, mas há correspondências inferiores menos coerentes.
a person outdoors	Muito boa: a combinação pessoa + ambiente externo aparece de forma consistente.
a child playing sports	Parcial: o melhor resultado é forte, enquanto outros satisfazem apenas partes da composição.
a person using an electronic device	Fraca/parcial: aparecem eletrônicos sem pessoa usando-os e pessoas sem ação clara.
a relaxing outdoor scene	Muito boa: cenas externas/naturais compatíveis com a ideia abstrata de relaxamento.
a happy social moment	Boa/parcial: captura felicidade e pessoas, mas nem todos os resultados expressam interação social clara.

4.5 CLIP, tokenização, padding e attention mask

O CLIP textual recebe input_ids e attention_mask. Padding completa sequências de comprimentos diferentes para batching; attention mask marca quais posições são conteúdo. A diferença para BERT

está principalmente no objetivo: BERT aprende representação linguística; CLIP aprende alinhamento imagem-texto por contraste.

5. A3 - CNN e transfer learning

5.1 download_dataset.py, clean_dataset.py e deduplicate_dataset.py

O fluxo começa baixando o dataset, identificando a pasta correta das sete classes e validando imagens. Em seguida, SHA-256 agrupa duplicatas exatas. A deduplicação global precisa acontecer antes do split para impedir que o mesmo conteúdo apareça em treino e teste.

No experimento, 1.803 imagens válidas foram reduzidas a 1.764 imagens únicas após remoção de 39 duplicatas. check_duplicates.py verifica novamente hashes e vazamento entre splits.

5.2 data_pipeline.py

O pipeline aplica RandomResizedCrop(224, scale 0,8-1,0), horizontal flip p=0,5, RandomRotation(10) e ColorJitter leve. Depois converte para tensor e normaliza com mean/std do ImageNet. Validação e teste usam somente resize + normalização.

Por que isso importa: augmentation ensina invariâncias. Mas transformações precisam respeitar a semântica. Rotação extrema, crop agressivo ou jitter excessivo poderiam transformar exemplos em imagens irreais ou remover pistas úteis.

5.3 models/pretrained_cnn.py

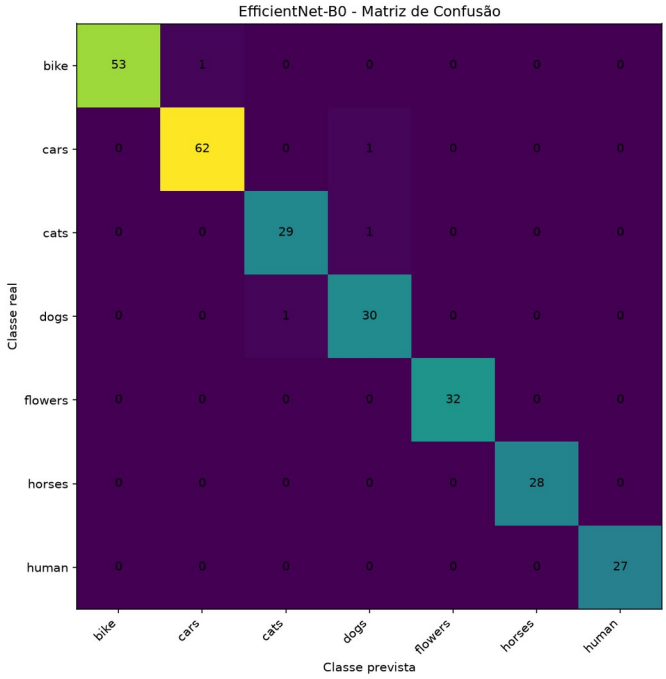
EcoVisionPretrainedCNN carrega EfficientNet-B0 com pesos ImageNet, identifica 1.280 features na camada final e troca o classifier para 7 classes. congelar_backbone desativa requires_grad no backbone; descongelar_modelo reativa os parâmetros.

```
features = 1280
num_classes = 7
Linear(1280, 7) -> 8.967 parâmetros (incluindo bias)
```

5.4 train.py

executar_fase abstrai cada fase de treinamento. Na feature extraction, apenas o head é atualizado por 2 epochs com LR 1e-3. No fine-tuning, todo o modelo é treinável por até 8 epochs com LR 1e-4. salvar_checkpoint preserva o melhor validation loss e gerar_graficos documenta o processo.

Resultado final: 98.49% accuracy e 98.50% F1 weighted. A matriz de confusão mostra apenas quatro erros em 265 imagens.



Exemplo visual - matriz de confusão da EfficientNet-B0.

6. A4 - Radiografias e GANs

6.1 prepare_dataset.py

O módulo baixa/localiza o COVID-19 Radiography Database, valida imagens, calcula SHA-256, remove duplicatas, seleciona quantidades exatas por classe e salva manifest. O conjunto final replica 840 Normal, 240 Pneumonia e 120 COVID.

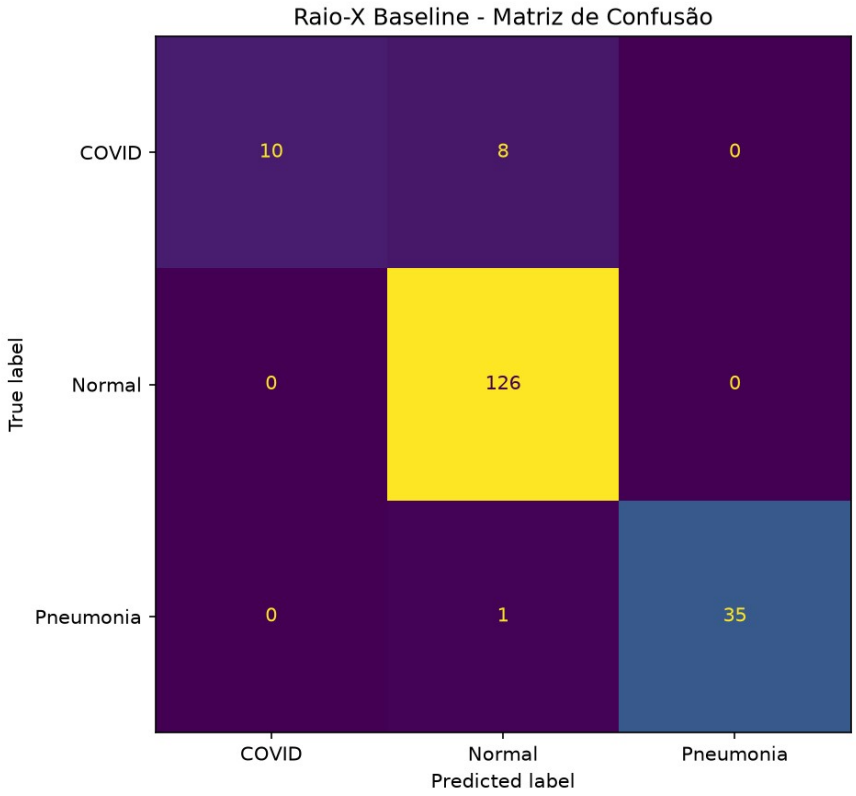
6.2 data_pipeline.py

O split estratificado 70/15/15 é fixado por seed 42. As augmentations são suaves: rotação 7 graus, RandomAffine com translate 0,03 e scale 0,95-1,05, brilho 0,05 e contraste 0,10. O cuidado existe porque transformações geométricas agressivas poderiam distorcer anatomia.

6.3 models/classifier.py e training/baseline.py

XRayClassifier carrega ResNet-18 ImageNet e troca fc 512->3. training/baseline.py usa AdamW 1e-4, weight decay 1e-4, CrossEntropy não ponderada, máximo 10 epochs e early stopping 3. A loss sem pesos foi mantida para que, no experimento sintético, a principal diferença fosse a adição de imagens geradas.

O baseline de teste obteve 95.00% accuracy, mas somente 55.56% de recall COVID. Essa divergência é um exemplo didático de por que accuracy não deve ser a única métrica em dados desbalanceados.



Exemplo visual - baseline: 8 casos COVID foram classificados como Normal.

6.4 gan_data.py

A GAN usa somente os índices de treino do classificador. As imagens são convertidas para grayscale 64x64 e normalizadas para [-1,1]. Isso garante que validação e teste não contaminem o treinamento generativo.

6.5 models/cgan.py - Generator

ConditionalGenerator concatena latent vector z (100 dimensões) e embedding da classe (100 dimensões). A representação 200x1x1 passa por camadas ConvTranspose2d, BatchNorm e ReLU até 64x64; Tanh produz a imagem final. ConvTranspose aumenta dimensão espacial e aprende como distribuir features na imagem.

```
z (100) + label_embedding (100)
-> 200 x 1 x 1
-> 512 x 4 x 4
-> 256 x 8 x 8
-> 128 x 16 x 16
-> 64 x 32 x 32
-> 1 x 64 x 64
-> Tanh
```

6.6 models/cgan.py - Discriminator

ConditionalDiscriminator transforma a classe em um mapa 64x64 e concatena com a imagem, criando 2 canais. Convoluções reduzem progressivamente a resolução e geram um logit. Não existe Sigmoid explícita porque BCEWithLogitsLoss incorpora a transformação de modo numericamente estável.

6.7 training/cgan_base.py

A cGAN base usa 20 epochs, batch 32, Adam com LR 2e-4 em G e D e betas (0,5, 0,999). WeightedRandomSampler equilibra a exposição às classes. O loop de D usa real=1 e fake=0; fake.detach() corta o grafo em G. Em seguida, G é atualizado tentando fazer D(fake) aproximar-se do target real.

calcular_diversidade_covid gera 32 COVID com ruídos diferentes e calcula o desvio padrão médio por pixel. É uma métrica auxiliar: baixa diversidade sugere collapse, mas valor alto não garante realismo.

6.8 training/cgan_mitigated.py

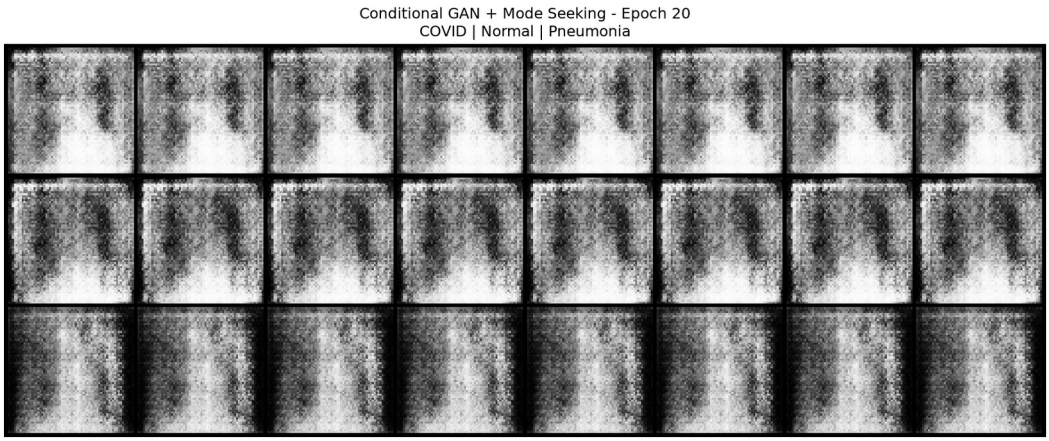
A primeira mitigação usa G LR 2e-4 e D LR 1e-4, real label 0,9 e instance noise que cai linearmente de 0,05 a 0. A intenção é impedir D de ficar excessivamente forte cedo. O equilíbrio D(real)/D(fake) melhorou, mas a diversidade final caiu para 0,0189.

6.9 training/cgan_diversity.py

A terceira versão adiciona configurar_gradientes para congelar temporariamente D durante o passo de G e calcular_mode_seeking_loss. Dois z distintos com a mesma classe são enviados a G. O score é a razão entre distância das imagens e distância dos latentes. Como queremos maximizar a razão, a loss adicionada é negativa.

```
score_ms = mean(|G(z1,y)-G(z2,y)|) / (mean(|z1-z2|) + eps)
loss_ms = -score_ms
loss_g = adversarial_loss + 1.0 * loss_ms
```

A diversidade final foi 0,0261, superior às versões anteriores, mas as grades visuais ainda mostraram imagens muito semelhantes. Isso é uma conclusão metodologicamente importante: a mitigação apresentou evidência parcial de melhoria, sem resolver completamente o collapse.



Exemplo visual - cGAN Mode-Seeking na época 20; as colunas permanecem muito semelhantes.

6.10 generation/generate_synthetic_covid.py

O módulo carrega apenas o Generator da versão Mode-Seeking, amostra 84 vetores z com label COVID, converte saídas de [-1,1] para [0,1], transforma grayscale em RGB e salva PNG 64x64. A validação confirma exatamente 84 arquivos, dimensões corretas e modo RGB.

6.11 training/synthetic.py e evaluation/synthetic.py

O novo treino concatena o subset real de treino com o dataset sintético. Validação e teste permanecem reais. A arquitetura, CrossEntropy e hiperparâmetros do baseline são mantidos. evaluation/synthetic.py calcula as mesmas métricas e compara automaticamente com o JSON do baseline.

Resultado: COVID recall caiu de 55.56% para 38.89% (-16,67 pp). Isso transforma o experimento em uma aula importante: augmentation só é útil se os novos exemplos aumentarem a informação relevante; redundância e artefatos podem piorar generalização.

7. Métricas - como interpretar

Métrica	Pergunta que responde	Cuidado
Accuracy	Quantas previsões totais foram corretas?	Pode esconder falhas em classe minoritária.
Precision	Quando o modelo prevê esta classe, quantas vezes acerta?	Alta precision não significa encontrar todos os positivos.
Recall	Entre todos os positivos reais, quantos foram encontrados?	Crítico quando falsos negativos são caros.
F1	Qual o equilíbrio harmônico entre precision e recall?	Depende do threshold/decisão do modelo.
Macro F1	Como o modelo performa tratando classes igualmente?	Útil em desbalanceamento.
Weighted F1	Como performa ponderando pelo suporte de cada classe?	Pode ficar dominado por classes grandes.
Confusion matrix	Quais classes estão sendo confundidas?	Exige olhar absoluto e relativo.

8. Problemas encontrados e o que aprendemos ao corrigir

Problema	Correção	Aprendizado
Dataset CNN contado como 3.606	Identificar raiz real das classes e limpar estrutura aninhada.	Conhecer estrutura física do dataset antes de usar rglob/recursão.
Duplicatas no dataset CNN	SHA-256 global antes do split.	Data leakage pode inflar métricas de modo silencioso.
CLIP API/projeção	Usar text_model/vision_model + projection e normalização L2 explicitamente.	Embeddings comparáveis precisam estar no espaço projetado e normalizado.
Threshold CLIP arbitrário	Analisar distribuição e sensibilidade 0,22-0,25.	Threshold de similaridade é específico do corpus e não é probabilidade.
cGAN com D dominante	LR assimétrico, label smoothing e instance noise.	Equilíbrio adversarial não é sinônimo de diversidade.
Mode collapse persistente	Mode-Seeking Regularization e análise visual.	Uma métrica única não substitui inspeção da qualidade.
Dados sintéticos pioraram recall	Comparação controlada no mesmo teste real.	Mais dados não garantem melhor generalização.
Código todo em src/	Separação por domínio + common/paths.py.	Arquitetura de projeto melhora manutenibilidade e estudo.

9. Mapa completo de módulos e funções

A tabela abaixo serve como índice do código organizado. Use-a para navegar do conceito para o arquivo correspondente.

VIT

Módulo	Classes	Funções de nível de módulo
src/vit/attention_maps.py	-	carregar_modelo, desnormalizar_imagem, encontrar_exemplo_correto, extrair_attention_map, visualizar_attention, main
src/vit/data_pipeline.py	-	localizar_dataset, criar_indices, criar_datasets, criar_dataloaders, main
src/vit/download_dataset.py	-	criar_diretorios, baixar_dataset, extrair_dataset, localizar_dataset, listar_classes, main
src/vit/eda.py	-	localizar_dataset, carregar_dataset, mostrar_informacoes, contar_imagens_por_classe, plotar_distribuicao, visualizar_exemplos, main
src/vit/evaluate_pretrained.py	-	carregar_modelo, obter_predicoes, calcular_accuracy_por_classe, salvar_matriz_confusao, main
src/vit/evaluate_scratch.py	-	carregar_modelo, obter_predicoes, calcular_accuracy_por_classe, gerar_matriz_confusao, main
src/vit/models/attention.py	ScaledDotProductAttention (__init__, forward); MultiHeadAttention (__init__, forward)	testar_scaled_dot_product_attention, testar_multi_head_attention
src/vit/models/pretrained_vit.py	EcoVisionPretrainedViT (__init__, forward, congelar_backbone, descongelar_modelo)	contar_parametros, testar_modelo
src/vit/models/transformer.py	FeedForward (__init__, forward); TransformerEncoderBlock (__init__, forward)	testar_feed_forward, testar_transformer_encoder
src/vit/models/vit.py	PatchEmbedding (__init__, forward); ViTEmbeddings (__init__, forward)	testar_patch_embedding, testar_vit_embeddings
src/vit/models/vit_model.py	EcoVisionViT (__init__, forward)	testar_ecovision_vit
src/vit/train_pretrained.py	-	contar_parametros, treinar_epoca, avaliar, salvar_modelo, executar_fase, gerar_graficos, main
src/vit/train_scratch.py	-	treinar_epoca, avaliar, salvar_checkpoint, salvar_historico, gerar_graficos, main

CLIP

Módulo	Classes	Funções de nível de módulo
src/clip/download_dataset.py	-	baixar_dataset, localizar_imagens, analisar_pastas, mostrar_diretorios, main
src/clip/prepare_corpus.py	-	localizar_dataset, calcular_sha256, localizar_usuarios, localizar_imagens_usuario, remover_duplicatas, selecionar_imagens, copiar_corpus, salvar_manifest, main

Módulo	Classes	Funções de nível de módulo
src/clip/semantic_ranking.py	-	carregar_clip, carregar_manifest, normalizar_embeddings, gerar_embeddings_texto, carregar_imagem, gerar_embeddings_imagens, calcular_similaridades, analisar_distribuicao, gerar_ranking, imprimir_ranking, salvar_ranking, salvar_grafico_top5, salvar_exemplos_top5, salvar_embeddings, salvar_configuracao, main
src/clip/semantic_search.py	-	normalizar_embeddings, carregar_dados, carregar_clip, gerar_embedding_consulta, buscar, imprimir_resultados, carregar_imagem, criar_slug, salvar_visualizacao, salvar_resultados, main
src/clip/threshold_analysis.py	-	normalizar_embeddings, gerar_embeddings_texto, main

CNN

Módulo	Classes	Funções de nível de módulo
src/cnn/check_duplicates.py	-	calcular_sha256, main
src/cnn/clean_dataset.py	-	imagem_valida, limpar_dataset
src/cnn/data_pipeline.py	-	localizar_dataset, criar_transforms, criar_datasets, criar_dataloaders, main
src/cnn/deduplicate_dataset.py	-	calcular_sha256, localizar_imagens, main
src/cnn/download_dataset.py	-	baixar_dataset, listar_diretorios, localizar_imagens, analisar_classes, main
src/cnn/evaluate.py	-	carregar_modelo, obter_predicoes, calcular_accuracy_por_classe, salvar_matriz_confusao, main
src/cnn/models/pretrained_cnn.py	EcoVisionPretrainedCNN (__init__, forward, congelar_backbone, descongelar_modelo)	contar_parametros, testar_modelo
src/cnn/train.py	-	treinar_epoca, avaliar, salvar_checkpoint, executar_fase, gerar_graficos, main

XRAY

Módulo	Classes	Funções de nível de módulo
src/xray/data_pipeline.py	-	localizar_dataset, criar_transforms, criar_datasets, contar_classes, criar_dataloaders, mostrar_distribuicao, main
src/xray/evaluation/baseline.py	-	avaliar, calcular_metricas, salvar_matriz_confusao, main
src/xray/evaluation/synthetic.py	-	avaliar, calcular_metricas, salvar_matriz_confusao, comparar_com_baseline, main
src/xray/gan_data.py	-	criar_transform_gan, criar_dataset_gan, criar_gan_loader, main
src/xray/generation/ generate_synthetic_covid.py	-	configurar_seed, carregar_generator, gerar_imagens, desnormalizar, salvar_imagens, salvar_preview, validar_imagens, main

Módulo	Classes	Funções de nível de módulo
src/xray/models/cgan.py	ConditionalGenerator (__init__, forward); ConditionalDiscriminator (__init__, forward)	main
src/xray/models/classifier.py	XRayClassifier (__init__, forward)	main
src/xray/prepare_dataset.py	-	baixar_dataset, calcular_sha256, localizar_dataset_principal, localizar_pasta_imagens, listar_imagens, imagem_valida, obter_imagens_unicas, selecionar_amostra, copiar_classe, salvar_manifest, main
src/xray/training/baseline.py	-	configurar_seed, treinar_epoca, validar, salvar_checkpoint, salvar_graficos, main
src/xray/training/cgan_base.py	-	configurar_seed, inicializar_pesos, criar_loader_balanceado, criar_amostras_fixas, salvar_amostras, calcular_diversidade_covid, salvar_curvas, main
src/xray/training/cgan_diversity.py	-	configurar_seed, inicializar_pesos, configurar_gradientes, criar_loader_balanceado, criar_amostras_fixas, obter_noise_std, adicionar_instance_noise, calcular_mode_seeking_loss, calcular_diversidade_covid, salvar_amostras, salvar_curvas, main
src/xray/training/cgan_mitigated.py	-	configurar_seed, inicializar_pesos, criar_loader_balanceado, criar_amostras_fixas, obter_noise_std, adicionar_instance_noise, calcular_diversidade_covid, salvar_amostras, salvar_curvas, main
src/xray/training/synthetic.py	-	configurar_seed, criar_datasets_com_sinteticas, criar_dataloaders_augmented, treinar_epoca, validar, salvar_checkpoint, salvar_graficos, main

10. Leitura do código arquivo por arquivo

Esta seção complementa o mapa de módulos. O objetivo é explicar o papel de cada arquivo na arquitetura organizada e o que procurar ao lê-lo no editor.

src/common/paths.py

Centraliza PROJECT_ROOT. Procure como Path(__file__) é percorrido para que módulos em diferentes profundidades encontrem data/, models/ e results/. Isso evita caminhos frágeis após reorganização.

src/vit/download_dataset.py

Responsável por criar diretórios, baixar/extrair RealWaste e localizar a pasta final. Estude como o código evita misturar download com treinamento.

src/vit/eda.py

EDA: contagem por classe, distribuição e exemplos. Serve para compreender o dataset antes de definir arquitetura e métricas.

src/vit/data_pipeline.py

Cria índices estratificados, aplica transforms e DataLoaders. Observe a separação entre train_transform e eval_transform.

src/vit/models/attention.py

Implementação matemática da attention. Leia primeiro o forward de ScaledDotProductAttention e depois o reshape/split/concat de MultiHeadAttention.

src/vit/models/transformer.py

Composição de attention + FFN + LayerNorm + residual. Observe que a ordem implementada é pre-norm.

src/vit/models/vit.py

Transforma pixels em tokens: Conv2d para patches, flatten/transpose, CLS token e positional embedding.

src/vit/models/vit_model.py

Monta o ViT completo e decide quando retornar attention weights. É o ponto de integração dos módulos menores.

src/vit/train_scratch.py

Treino do zero. Compare treinar_epoca e avaliar para perceber o que muda entre model.train() e model.eval().

src/vit/attention_maps.py

Interpretação espacial. Siga a cadeia: carregar modelo -> encontrar exemplo -> extrair CLS attention -> interpolar -> sobrepor.

src/vit/models/pretrained_vit.py

Wrapper do timm. Estude a substituição do head e os métodos de freeze/unfreeze.

src/vit/train_pretrained.py

Mostra um padrão reutilizável de duas fases: feature extraction e fine-tuning.

src/clip/prepare_corpus.py

Pipeline de curadoria: usuários -> arquivos -> hash -> deduplicação -> amostragem -> manifest.

src/clip/semantic_ranking.py

Pipeline CLIP de ponta a ponta: carregar modelo, codificar textos/imagens, normalizar, calcular matriz de similaridades, ranquear e salvar evidências.

src/clip/threshold_analysis.py

Experimento pequeno para entender sensibilidade do threshold sem misturar com o pipeline principal.

src/clip/semantic_search.py

Reutiliza embeddings visuais e calcula apenas o embedding da consulta; é um exemplo de arquitetura eficiente para retrieval.

src/cnn/download_dataset.py

Baixa e inspeciona a estrutura original. Útil para entender por que uma raiz errada duplicava a contagem.

src/cnn/clean_dataset.py

Valida arquivos de imagem e materializa uma árvore limpa de classes.

src/cnn/deduplicate_dataset.py

Agrupa por SHA-256 antes do split. Estude por que essa ordem previne leakage.

src/cnn/check_duplicates.py

Auditoria independente para confirmar ausência de hashes repetidos/cross-split.

src/cnn/data_pipeline.py

Augmentation, normalização ImageNet, split estratificado e DataLoaders.

src/cnn/models/pretrained_cnn.py

Wrapper da EfficientNet-B0; troca classifier e expõe freeze/unfreeze.

src/cnn/train.py

Treino em fases. executar_fase é a função central para reduzir duplicação entre head-only e fine-tuning.

src/cnn/evaluate.py

Carrega checkpoint, obtém predições, calcula métricas por classe e salva matriz de confusão.

src/xray/prepare_dataset.py

Monta exatamente 1.200 imagens a partir da base maior, validando hash e quantidades por classe.

src/xray/data_pipeline.py

Define augmentation médico leve, split estratificado e contagem das classes.

src/xray/models/classifier.py

ResNet-18 pré-treinada com fc 512->3.

src/xray/training/baseline.py

Baseline controlado; registra accuracy, macro F1 e COVID recall e salva melhor validation loss.

src/xray/gan_data.py

Isola apenas o split de treino para a GAN, converte para grayscale 64x64 e [-1,1].

src/xray/models/cgan.py

Arquiteturas condicionais de G e D. Compare como a classe entra nos dois lados do jogo adversarial.

src/xray/training/cgan_base.py

Implementação adversarial base e primeira evidência de mode collapse.

src/xray/training/cgan_mitigated.py

Mantém arquitetura e altera dinâmica: D mais lento, label smoothing e instance noise.

src/xray/training/cgan_diversity.py

Adiciona freeze explícito de D durante G e a regularização Mode-Seeking.

src/xray/generation/generate_synthetic_covid.py

Transforma o Generator em ferramenta de geração offline: amostra z, salva 84 PNGs e valida formato.

src/xray/training/synthetic.py

ConcatDataset real + sintético somente no treino. Mantém validação/teste reais.

src/xray/evaluation/baseline.py

Métricas e confusion matrix do baseline.

src/xray/evaluation/synthetic.py

Repete as métricas e produz comparação automática com baseline.

11. Comandos de execução

```
# ViT
python -m src.vit.train_scratch
python -m src.vit.evaluate_scratch
python -m src.vit.train_pretrained
python -m src.vit.evaluate_pretrained
python -m src.vit.attention_maps

# CLIP
python -m src.clip.prepare_corpus
python -m src.clip.semantic_ranking
python -m src.clip.threshold_analysis
```

```
python -m src.clip.semantic_search

# CNN
python -m src.cnn.train
python -m src.cnn.evaluate

# Raio-X
python -m src.xray.training.baseline
python -m src.xray.evaluation.baseline
python -m src.xray.training.cgan_base
python -m src.xray.training.cgan_mitigated
python -m src.xray.training.cgan_diversity
python -m src.xray.generation.generate_synthetic_covid
python -m src.xray.training.synthetic
python -m src.xray.evaluation.synthetic
```

12. Glossário rápido

Termo	Definição
Backbone	Parte principal do modelo que extrai features.
Classification head	Camada final que transforma features em logits de classes.
Embedding	Representação vetorial de uma entrada.
Logit	Score bruto antes de softmax/sigmoid.
Epoch	Uma passagem completa pelo conjunto de treino.
Batch	Subconjunto processado em uma atualização.
Learning rate	Tamanho do passo do otimizador.
Weight decay	Regularização que desencoraja pesos grandes.
Dropout	Desativa aleatoriamente unidades no treino para regularização.
BatchNorm	Normaliza ativações usando estatísticas do batch.
LayerNorm	Normaliza features dentro do token/amostra; muito usada em Transformers.
Residual connection	Soma entrada original à saída de uma transformação.
Patch	Região da imagem tratada como token pelo ViT.
CLS token	Token aprendível usado como resumo global para classificação.
Positional embedding	Informação de posição adicionada aos tokens.
Attention head	Subespaço independente de atenção.
Cosine similarity	Similaridade angular entre vetores.
Zero-shot	Aplicar conceitos/tarefas sem treinamento supervisionado específico naquele conjunto de classes.
Fine-tuning	Atualizar pesos pré-treinados no novo domínio.
Feature extraction	Congelar backbone e treinar apenas o head.
GAN	Generator e Discriminator treinados adversarialmente.
Mode collapse	Generator produz pouca diversidade apesar de diferentes z.
Recall	Proporção dos positivos reais corretamente detectados.
Data leakage	Informação do teste entra direta ou indiretamente no treino.
Domain shift	Distribuição em produção difere da distribuição de treino.

13. Perguntas para revisão

1. Por que dividir QK^T por sqrt(d_k) no Scaled Dot-Product Attention?
2. O que muda entre uma head e Multi-Head Attention?
3. Por que o ViT precisa de positional embeddings?
4. Por que o ViT pré-treinado superou o scratch neste dataset?
5. Qual a diferença prática entre feature extraction e fine-tuning?
6. Por que CLIP consegue buscar imagens por texto sem treinar um novo classificador?
7. Por que 0,22 no CLIP não pode ser interpretado como 22% de probabilidade?
8. Qual a função de padding e attention mask em um encoder textual?
9. Por que duplicatas precisam ser removidas antes do split?
10. Por que accuracy pode ser enganosa no caso de COVID?
11. O que fake.detach() faz no treinamento da GAN?
12. Como label smoothing e instance noise tentam estabilizar D?
13. Como a Mode-Seeking loss tenta combater mode collapse?
14. Por que diversidade alta na época 1 não significou boa qualidade?
15. Por que adicionar 84 imagens sintéticas piorou o recall?
16. Como você redesenharia a validação para um sistema clínico real?

14. Resumo final de resultados

Experimento	Resultado principal	Leitura
ViT scratch	55,26% accuracy	Arquitetura funciona, mas dados limitam aprendizado do zero.
ViT pré-treinado	86,54% accuracy	Pré-treinamento adicionou +31,28 pp.
CLIP ADS-16	24 conceitos / 8 buscas	Boa semântica direta; composições complexas são parciais.
EfficientNet-B0	98,49% accuracy	Transfer learning + deduplicação produziram forte generalização no teste.
X-Ray baseline	95% accuracy; 55,56% COVID recall	Accuracy global esconde falha na minoria.
cGAN Mode-Seeking	diversidade final 0,0261	Melhora parcial; collapse visual persiste.
X-Ray + sintéticos	38,89% COVID recall	Dados sintéticos de baixa diversidade pioraram a tarefa downstream.

15. Referências essenciais para estudo

- Vaswani et al. (2017) - Attention Is All You Need - https://arxiv.org/abs/1706.03762
- Devlin et al. (2018) - BERT - https://arxiv.org/abs/1810.04805
- Dosovitskiy et al. (2020) - Vision Transformer - https://arxiv.org/abs/2010.11929
- Touvron et al. (2020) - DeiT - https://arxiv.org/abs/2012.12877
- Liu et al. (2021) - Swin Transformer - https://arxiv.org/abs/2103.14030
- Radford et al. (2021) - CLIP - https://arxiv.org/abs/2103.00020

- Tan e Le (2019) - EfficientNet - https://proceedings.mlr.press/v97/tan19a.html
- Goodfellow et al. (2014) - GAN - https://arxiv.org/abs/1406.2661
- Mao et al. (2019) - Mode Seeking GAN - https://openaccess.thecvf.com/content_CVPR_2019/html/Mao_Mode_Seeking_Generative_Adversarial_Networks_for_Diverse_Image_Synthesis_CVPR_2019_paper.html
- RealWaste - UCI Machine Learning Repository - https://archive.ics.uci.edu/dataset/908/realwaste

Cobertura final da rubrica

Cobertura final da rubrica: 26 de 26 critérios possuem evidência explícita nos notebooks, resultados e relatório acadêmico. Os quatro notebooks foram executados integralmente no Google Colab com GPU Tesla T4, além da revalidação local dos checkpoints após a reorganização do projeto.

Para revisão antes da apresentação, priorize: (1) explicar por que accuracy pode esconder baixo recall; (2) diferenciar feature extraction de fine-tuning; (3) explicar Q/K/V e o papel do positional embedding; (4) distinguir embedding CLIP de probabilidade; (5) explicar o adversarial loop da GAN e por que mode collapse ocorreu; (6) interpretar o resultado negativo da augmentation sintética sem generalizá-lo para todas as GANs médicas.